UGREEN Hackathon 2026 on AI NAS — Project Proposal

Project: care-circle

Track: Automate Your Workflow

Round 1 submission

---

1. What the Skill does (one sentence)

care-circle turns one folder of medication photos, clinic letters and voice memos into a dated, source-cited doctor-visit brief and a sibling update, computed on the NAS.

2. Who is the target user and problem?

Adult children caring for an ageing parent. Evidence is scattered across pill-bottle photos, discharge summaries and unplayed voice memos, so every appointment is a scramble.

3. What is your proposed solution?

A local care brief that reads each modality with the matching UGOS expert and writes two dated documents: a source-cited brief covering current medications with source files, label-versus-report conflicts, recent results, changes since the last visit, questions for the doctor and an unreadable list; and a five-line sibling update a family can share. It never diagnoses or advises dosage.

4. What is the Core User Flow?

The user names a care folder, the parent's name and the appointment date. The file expert enumerates and deduplicates the folder. The photo expert reads medication labels. The document expert extracts clinic letters and compares each medication against its label. The knowledge expert transcribes voice memos. Both are approved, written back and tagged at source.

5. What substantive difference does your idea offer compared with existing tools or conventio

Existing tools are static inventories, expiry diffs or single-modal cloud assistants. None reads a pill bottle with on-device vision, transcribes the parent's voice, extracts the clinic letter and binds every claim to its source file. Health data never leaves the NAS, and the brief is written back with sources tagged in place.

6. Which UGREEN AI NAS capabilities will be used, and at which step in the workflow?

Step 2: `ug-filesystem find`, `ug-filesystem ls`, `ug-filemgr scan-dedupe` (file-agent). Step 3: `ug-photo vision-ocr`, `ug-photo vision-vlm` (photo-agent). Steps 4-5: `ug-document metadata`, `page-count`, `extract` schema `medical-report`, `qa` intent `comparison` (document-agent). Step 6: `ug-knowledge audio-transcription` (knowledge-agent). Step 9: `ug-filesystem mkdir`. Step 10: two `ug-knowledge save-text` calls. Step 11: `ug-filemgr tag-get-by-name`, `tag-add`, `tag-add-file`.

7. What was your technical approach?

Pure Markdown orchestration. It delegates only through `delegate --target` to the four registered experts in a fixed order, passing each step's real result downstream. Read steps are ungated; the brief approval, folder creation, write and tags are approval-gated. Inputs: the folder, the parent name, the date. The NAS local model performs vision, extraction and transcription; nothing leaves the device.

8. What is the current development status?

Implemented: the full twelve-step orchestration, the brief structure, the label-versus-report conflict

check, the unreadable list, approval handling, and a static checker pass with zero errors and zero warnings. Pending NAS validation: on-device vision, extraction and transcription behaviour, tag creation and write approval behaviour.

9. What is your final validation plan?

Core case 1: three medication photos, one clinic letter and one voice memo produce a brief with every claim cited and every unreadable file listed. Core case 2: a second run with an added earlier letter shows a correct change since the last visit. Edge case: one blurry photo and no documents completes with an empty medication section and an explicit unreadable entry.

10. Are there any known limitations of your Project?

Vision reading of curved or reflective pill labels can fail; unreadable labels are reported, never guessed. Medication detail is only as good as the source documents. A label-versus-report conflict is reported, not resolved. Tag creation needs gateway approval. No runtime third-party dependencies; the only development tool is the UGREEN Uliya skill kit.

Team and consent

Entrant: Philip Jon Ji Ciuca (individual entry). All members consent to the official rules and confirm the Skill and its core workflow were developed during the hackathon. No previously developed Skill or submission material is reused.

Third-party components

The Skill has no runtime third-party dependencies. The Uliya Skill Creation Tool (uliya-skill-kit v0.7.3) was used at development time only and is not distributed inside the Skill package.